

Lab 3: Architecture Selection & Justification

Airport Lost Luggage Claim & Tracking Portal

Varun Rao | SRN: PES1UG24CS520 | Section I

We chose Microservices Architecture for the Airport Lost Luggage Claim & Tracking Portal.

Reason 1

Not all parts of this system need the same performance. Matching a claim against scan logs has to happen fast (under 200ms per NFR-001), since it's a search across every terminal's data and the passenger is waiting. Notifications, on the other hand, can take a bit longer without anyone caring. Splitting these into separate services lets us tune the Baggage Matching Service for speed without dragging everything else along or over-engineering parts that don't need it.

Reason 2

The system needs 99.9% uptime (NFR-002), and a monolith makes that harder since one bug anywhere can take the whole thing down. With microservices, if the Notification Service crashes or lags, passengers can still file claims and check status normally, because that part doesn't depend on notifications working. Each piece can fail on its own without taking the rest down with it.

Security Advantage

Since each service only talks to the others through specific defined interfaces (like the Notification Request or Match Request interfaces), a problem in one service doesn't automatically expose everything else. Nobody can reach the database directly except through the service that's actually responsible for that data, so a compromised service doesn't get a free pass to everyone else's records.

Performance Benefit

Because the Baggage Matching Service is its own component, it can be scaled or sped up independently — caching, extra instances, whatever's needed — to keep hitting that 200ms target as claim volume grows. That kind of targeted scaling would be messier in a single combined system, where you'd end up scaling everything together just to speed up the one part that actually needs it.